

First Come, First Served (FCFS)

FCFS serves processes in arrival order. Once a process starts, it keeps the CPU until it finishes.

1. Model

The visualizer uses one CPU and measures time in whole ticks. Processes only need CPU service; they do not block for I/O. A process's **service time** is the total number of CPU ticks it needs. Each tick of execution reduces its remaining service by one. Switching processes takes no time.

A future process has not arrived yet. A ready process has arrived but is not running or finished. In a queue such as [B, C], B is at the front. The short examples under the rules are separate scenarios; all times are in ticks.

2. Scheduling rules

Rule 1. Join the back of the queue on arrival

New processes enter at the tail of the ready queue. Processes that arrive together enter in their input order. A process already waiting stays ahead of a new arrival.

Explanation. Arrival determines a process's place in line. Once it is waiting, later arrivals cannot move ahead of it. Input order settles simultaneous arrivals because neither process arrived first.

Example. B is already waiting. C and A arrive together, with C listed before A in the input. Before dispatch, the queue is [B, C, A]. A's name does not give it an earlier position.

Rule 2. Run the process at the front

When the CPU is free, remove the queue head and run it. Service time and process ID do not affect this choice.

Explanation. FCFS does not search the queue for a shorter job. The process that has reached the front gets the next turn, however much CPU service it needs.

Example. The queue is [B, C]. B needs five ticks and C needs one. Select B. C remains in the ready queue while B runs.

Rule 3. Let the running process finish

FCFS is **non-preemptive**: an arrival cannot take the CPU away from the running process.

Explanation. Arrivals change the waiting queue, not the current CPU assignment. There is no time slice that forces the running process to give up the CPU.

Example. A starts at time 0 and needs five ticks. B arrives at time 1 needing one tick. A continues until time 5, then B runs from 5 to 6. B's short service time does not interrupt A.

Rule 4. Complete a process when its remaining service reaches zero

Remove the finished process before selecting another one. The next process may start at the same boundary.

Explanation. Completion is recorded at the end of the final tick. There is no extra tick for removing a process or switching to the next one. Arrivals at this boundary join behind processes already waiting.

Example. A executes its last tick from 4 to 5. B is waiting, and C arrives at time 5. Remove A, append C to form [B, C], then dispatch B at time 5. A never returns to the queue.

Rule 5. Idle only when nothing is ready

If the CPU and ready queue are empty, wait for the next arrival. Stop when every process has finished. An empty queue does not interrupt a process already running.

Explanation. An empty ready queue means that nobody is waiting; the CPU may still be busy. The CPU becomes idle only when there is also no running process. A future process cannot run before its arrival time.

Example. A arrives at time 0 with two service ticks. B arrives at time 5 with one. A finishes at time 2, so the CPU is idle from 2 to 5. B starts immediately at time 5 and finishes at time 6.

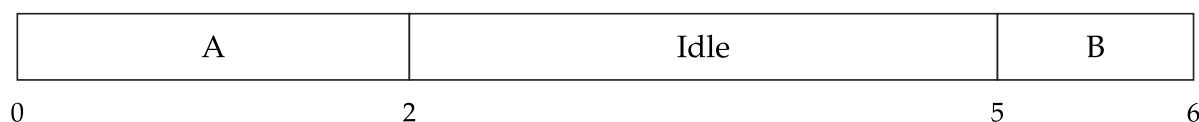


Figure 1. CPU schedule (time in ticks).

3. Events at a tick boundary

At time t , first account for execution during the preceding tick. Then:

1. Remove the running process if it has finished.
2. Add all arrivals at t to the queue tail, in input order.
3. If the CPU is free, dispatch the queue head.

The completion-and-arrival example under Rule 4 follows this order. B keeps its place ahead of C, and the CPU changes from A to B without an idle tick.

4. Worked example

The processes are entered in the order A, B, C. All times are in ticks.

<i>Process</i>	<i>Arrival</i>	<i>Service</i>
A	0	5
B	1	2
C	1	1

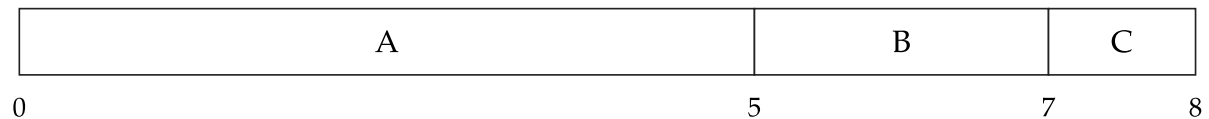


Figure 2. CPU schedule (time in ticks).

A starts at time 0. B and C arrive at time 1, but neither can interrupt it. At time 5, B runs before C because B was listed first among the simultaneous arrivals. The shorter process C still waits until time 7.